PII PROTECTION CASE STUDY

Detailed system architecture + member-wise implementation handbook

NAMED TEAM AND REVISED DIVISION OF WORK

Member 1: Ayush Sinha - ingestion and OCR

Member 2: Avishi Sharma - PII detection and detection metrics

Member 3: Yash - redaction, verification and release gate

Member 4: Nancy - PowerPoint + lightweight synthetic demo-fixture generator

Member 5: Tirth - orchestration, UI, audit/report assembly and integration

ROSTER NOTE

Anshika was also named alongside Member 5 earlier. This guide follows the latest correction that Member 5 is Tirth. If Anshika is also part of the group, confirm whether she is a sixth contributor or should replace Tirth.

SYSTEM OBJECTIVE

Build a privacy-first document pipeline that validates files, extracts text, detects and classifies PII, redacts it, verifies the saved output, blocks unsafe releases, and produces a privacy-conscious report. Every member can develop and test independently using shared contracts and mock data.

1. Revised ownership and workload balance

Nancy has two deliverables: a **small, independent demo-fixture coding task** and the PowerPoint. Her coding scope is deliberately lighter than the four technical modules because slide design, evidence collection and rehearsal are substantial work.

Member / owner	Main ownership	Added / bounded work	Must not own
1 - Ayush Sinha	Ingestion, extraction, OCR	Synthetic fixtures; extraction completeness and structure checks	Detection, redaction, UI
2 - Avishi Sharma	PII detection and classification	Ground truth, precision/recall/F1 and false-negative analysis	Parsing, redaction, UI
3 - Yash	Redaction, verification, release gate	Leakage tests, saved-artifact checks and structure preservation	Detection internals, UI
4 - Nancy	PowerPoint and presentation	Small synthetic demo-fixture generator, manifest and tests	Core detection/redaction, metric framework, orchestration
5 - Tirth	Orchestration, UI, audit/report, integration	Cross-module errors, workflow states and end-to-end tests	Internal algorithms of Members 1-3

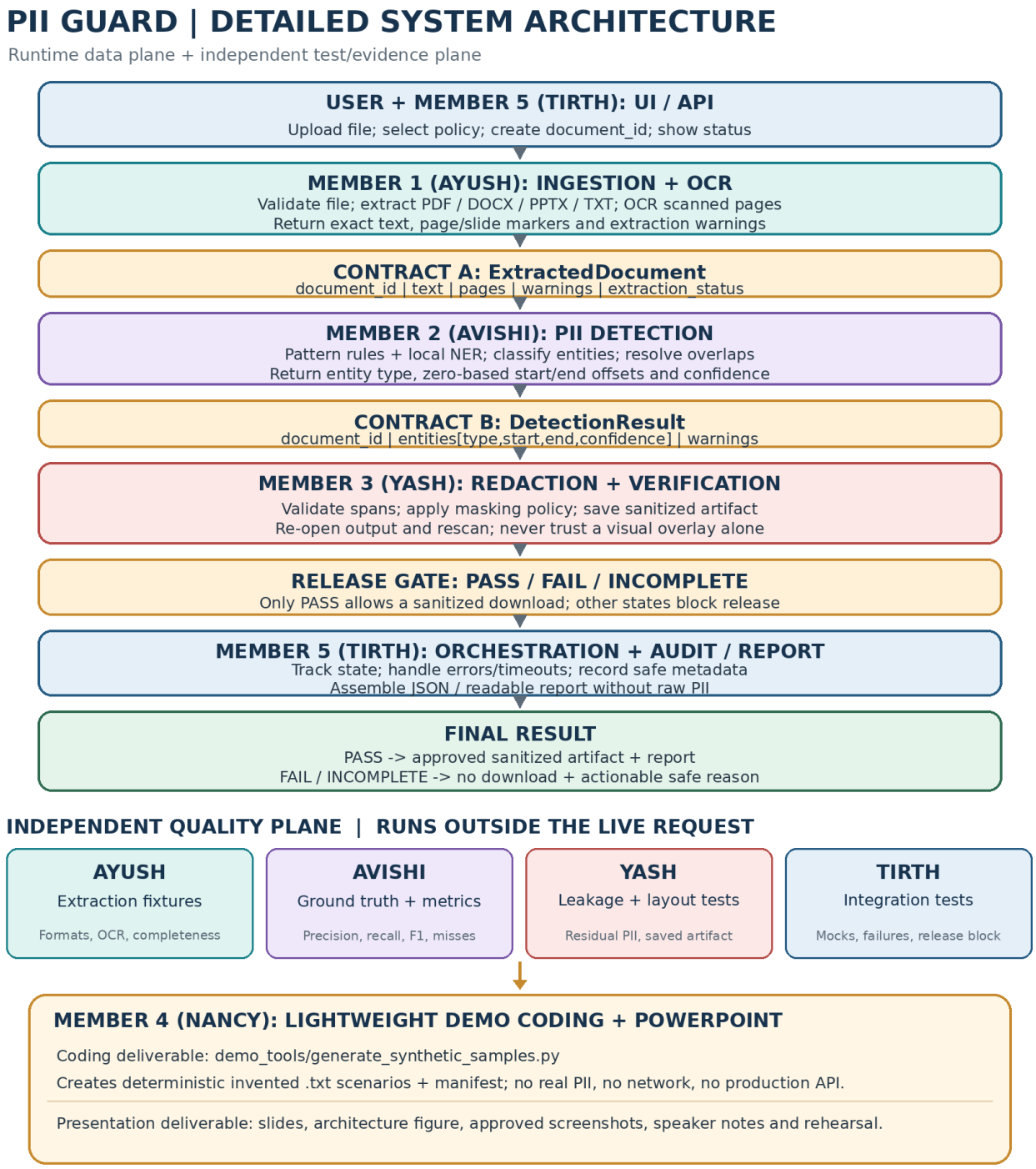
Nancy's intended workload: roughly 75% presentation / 25% lightweight coding is a planning guideline, not a score. She should not be assigned a second production module or a major evaluation framework.

Why this split works: every technical owner has a substantial production responsibility. Nancy's helper produces safe, repeatable demo inputs for the presentation and can be developed without waiting for the application pipeline.

Privacy default: all sample names, addresses, emails and identifiers must be invented. Do not use real personal documents or real credentials.

3. Complete runtime and team-support architecture

The main path is the live document-processing pipeline. The lower quality plane runs separately. Nancy's small fixture generator creates safe demo inputs but does not sit inside the production redaction path.



Boundary: Nancy's fixture generator supports demos and presentation evidence; it is NOT part of the live redaction path.

Trust rule: raw files stay private; use synthetic fixtures; only the release gate may approve downloads.

4. Architecture details and trust boundaries

Runtime data plane. One request passes through validation, extraction, PII detection, redaction, verification, then safe report assembly. Each stage returns a typed result or a structured error. No downstream stage should scrape another module's console logs.

Independent quality plane. Unit tests and measurements run outside the live request. Ayush owns extraction quality, Avishi owns detection metrics, Yash owns leakage/structure verification, Tirth owns cross-module integration, and Nancy supplies a small repeatable synthetic demo pack and slide evidence.

Storage plane. Keep uploaded originals in short-lived, access-controlled temporary storage. Keep sanitized output separate. Define retention and deletion behavior. Audit records contain metadata only. Clean temporary files on success and failure.

Component	Purpose	Owner / rule
UI / API / orchestrator	Accept file and policy; assign document ID; track state; route calls.	Tirth; not the owner of ingestion, detection or redaction internals.
Shared schemas	Versionable input/output contract for each module.	All agree once; contract changes go through reviewed PRs.
Original/quarantine storage	Hold source only during processing.	Tirth integrates lifecycle; never expose original download by default.
Sanitized-output storage	Store verified artifact separately.	Yash defines verification result; Tirth serves only if release_allowed=true.
Audit/report	Safe traceability without source text.	Tirth records status, counts, timestamps, versions; no raw PII.
Test/demo assets	Synthetic text cases, annotations, expected results.	Members 1-3 own their test data; Nancy owns only the presentation demo fixture pack.
Nancy's demo helper	Create deterministic synthetic .txt examples + manifest.	Offline helper; no network, no real PII, no production API dependency.

Release states: RECEIVED → VALIDATED → EXTRACTED → DETECTED → REDACTED → VERIFIED → RELEASED. A technical failure becomes FAILED; unsafe output becomes BLOCKED; missing required checks become INCOMPLETE. All states except verified PASS prevent download.

5. Member 1 - Ayush Sinha: ingestion, extraction and OCR

Mission: turn supported files into structured text while preserving enough location/structure metadata for accurate downstream detection and output verification. Ayush also owns extraction-quality tests and synthetic input fixtures.

Path	Responsibility
src/pii_guard/ingestion/validator.py	Validate extension and actual file signature where feasible, size/page limits, empty/corrupt files and unsupported formats.
src/pii_guard/ingestion/extractor.py	Extract PDF, DOCX, PPTX and TXT content using format-specific adapters; preserve page/slide markers and reading order as well as supported libraries allow.
src/pii_guard/ingestion/ocr.py	Detect scanned/image-based pages, call the agreed OCR library, capture confidence/warnings and flag unreadable pages.
src/pii_guard/ingestion/schemas.py	Define/import ExtractedDocument schema or shared schema types.
tests/test_ingestion.py	Test each supported format, OCR-needed documents, malformed inputs, warnings, offsets and extraction completeness.
docs/ingestion.md	Setup dependencies, formats supported, limitations, test command and examples.

Implementation recipe

- **Step 1 - inspect first:** check the existing repository, Python version, requirements and import conventions before adding files.
- **Step 2 - validator:** reject zero-byte/corrupt/oversized files with a standard error object. Do not trust the filename extension alone.
- **Step 3 - adapters:** implement one extractor per format but return the same ExtractedDocument schema from every adapter.
- **Step 4 - OCR:** identify pages with little/no embedded text and run OCR only as needed. If OCR fails or confidence is low, return a partial/incomplete status and warning.
- **Step 5 - stable text:** normalize only documented whitespace/encoding details and do so before handing text to Member 2. Do not alter the text after detection offsets are computed.
- **Step 6 - fixtures:** create synthetic files containing invented names, emails, addresses and phones, plus one malformed file and one scanned/image-only sample if feasible.
- **Step 7 - extraction-quality measure:** compare extracted text with fixture ground truth and define structure preservation (e.g., page, heading, paragraph or table retention) before reporting a percentage.

Public API and output

```
def extract_document(file_path: str) -> ExtractedDocument:
    """Validate file, extract text and return structured status/warnings."""

ExtractedDocument = {
  "document_id": "doc-001",
  "source_type": "pdf|scanned_pdf|docx|pptx|txt",
  "text": "exact text used by downstream offsets",
  "pages": [],
  "warnings": [],
  "extraction_status": "success|partial|failed"
}
```

Acceptance criteria

- All agreed formats work, or return explicit unsupported/error status.
- Output schema always includes status and warnings; OCR failures do not silently become success.
- Tests are runnable without Members 2-5 and do not require API credentials.
- Extraction completeness and known structure/OCR limitations are written in docs/ingestion.md.

Test and Nancy evidence handoff

```
pytest -q tests/test_ingestion.py
```

Send Nancy: supported-format table, a synthetic extracted-text screenshot, extraction test outcome/sample size, and one known OCR limitation.

6. Member 2 - Avishi Sharma: detection, classification and evaluation

Mission: locate PII in extracted text, classify it consistently and return exact offsets. Avishi absorbs the earlier evaluation responsibility: synthetic ground truth, precision/recall/F1 and false-negative analysis.

Path	Responsibility
src/pii_guard/detection/pattern_detector.py	Regex/pattern rules for emails, phones, IP addresses and specifically agreed identifier formats.
src/pii_guard/detection/ner_detector.py	Optional local NER adapter for person names, locations and address-like entities.
src/pii_guard/detection/classifier.py	Normalize taxonomy labels, confidence, duplicates and conflicting/overlapping detections.
src/pii_guard/detection/detector.py	Stable public detect_pii() function.
evaluation/ground_truth.jsonl	Synthetic annotated text and expected entity type/start/end spans.
evaluation/metrics.py	Calculate precision, recall, F1 and false-negative summaries.
tests/test_detection.py	Patterns, punctuation/spacing, offsets, duplicates, overlap, ambiguity and no-PII input.

Implementation recipe

- **Step 1 - taxonomy:** start with PERSON, EMAIL, PHONE, ADDRESS, GOVERNMENT_ID, FINANCIAL_IDENTIFIER and IP_ADDRESS. Implement only identifier formats that are in the agreed use case.
- **Step 2 - deterministic rules:** write tested patterns for predictable formats and validate candidates where a validation rule exists. Do not classify every long number as an ID.
- **Step 3 - contextual detection:** add a local NLP/NER model behind an adapter if useful and available. Keep raw documents away from public LLM endpoints; document fallback behavior if the model is unavailable.
- **Step 4 - spans:** return type, start, end and confidence. Offsets are zero-based and end-exclusive in Member 1's exact text.
- **Step 5 - overlap:** establish deterministic policy, e.g. prefer a specific validated ID over a generic number. Keep a warning for ambiguous matches.
- **Step 6 - ground truth:** annotate synthetic text by hand; include negative examples, punctuation/spacing variants and confusing near-matches.
- **Step 7 - metrics:** use exact-span plus entity-type matching unless the report specifies a different rule. Guard against zero denominators and state sample size.
- **Step 8 - false negatives:** inspect missed entities and describe limitations. Do not tune only against a tiny sample of examples.

Public API and output

```
def detect_pii(document: ExtractedDocument) -> DetectionResult:
    """Return classified entities and exact offsets into document.text."""
```

```
DetectionResult = {
  "document_id": "doc-001",
  "entities": [
    {"type": "EMAIL", "start": 12, "end": 29, "confidence": 0.98}
  ],
  "warnings": []
}
```

Metric definitions

```
precision = TP / (TP + FP)
recall    = TP / (TP + FN)
F1        = 2 * precision * recall / (precision + recall)
```

Acceptance criteria

- The detection result has validated offsets and standard entity labels.
- Synthetic ground-truth evaluation is repeatable and includes no-PII cases.
- Overall and per-category metrics are reported where sample size allows.
- False negatives and detector limitations are documented; no raw real-world PII is in fixtures or logs.

Test and Nancy evidence handoff

```
pytest -q tests/test_detection.py
```

Send Nancy: metric table including dataset size and matching rule; one synthetic detection example; and two or three known error/limitation notes.

7. Member 3 - Yash: redaction, verification and release gate

Mission: remove the spans returned by detection, inspect the actual saved artifact and make the release decision. Yash also owns leakage testing, redaction-effectiveness checks and structure-preservation tests.

Path	Responsibility
src/pii_guard/redaction/policy.py	Map PII categories to replacement tokens and define uncertain-case behavior.
src/pii_guard/redaction/redactor.py	Validate spans, handle overlaps and apply redactions deterministically.
src/pii_guard/redaction/verifier.py	Rescan redacted output; check known synthetic values; inspect exported file when supported.
src/pii_guard/redaction/gate.py	Return PASS/FAIL/INCOMPLETE and block release when required verification fails or cannot run.
tests/test_redaction.py	Leakage, invalid spans, overlaps, exact removal, non-PII preservation, output artifact tests and blocked release.
docs/security.md	Threat model, safe-redaction limitations, verification behavior and structure score method.

Implementation recipe

- **Step 1 - replacement policy:** use placeholders such as [REDACTED_EMAIL] or [REDACTED_PERSON]. Never include removed values in logs or normal audit records.
- **Step 2 - validate offsets:** enforce 0 <= start <= end <= len(text). Reject invalid spans and define how nested/overlapping ranges are resolved.
- **Step 3 - replace safely:** process text spans from highest offset to lowest so earlier offsets do not shift during replacement.
- **Step 4 - text verification:** search for known synthetic source values and apply independent residual checks. “No findings” is not proof that every possible PII value was detected.
- **Step 5 - artifact verification:** save output, reopen it and extract text from the saved file. A drawn black overlay is not true PDF redaction when underlying text remains selectable or extractable.
- **Step 6 - fail-closed gate:** required check fails or is unavailable => release_allowed=false. Member 5 must never bypass the gate.
- **Step 7 - structure testing:** define measure first: retained headings, paragraphs, tables, page/slide boundaries or a weighted score. Compare synthetic source/output fixtures.
- **Step 8 - explain trade-offs:** redaction can change line length and pagination; report formatting limits rather than claiming unchanged layout.

Public API and output

```
def redact_and_validate(document, detection_result) -> RedactionResult:
    """Apply policy, verify saved output and return a fail-closed decision."""
```

```
RedactionResult = {
  "document_id": "doc-001",
  "redacted_text": "Contact [REDACTED_EMAIL]",
  "redactions_applied": 1,
  "verification_status": "PASS|FAIL|INCOMPLETE",
  "release_allowed": False,
  "findings": []
}
```

Acceptance criteria

- Known synthetic PII is removed from the output and, for supported formats, from the saved artifact’s extractable content.
- Invalid spans, failed checks and incomplete verification block release.
- The test suite checks non-sensitive content preservation and records limitations.
- The gate has an unambiguous PASS/FAIL/INCOMPLETE meaning.

Test and Nancy evidence handoff

```
pytest -q tests/test_redaction.py
```

Send Nancy: synthetic before/after screenshot, leakage test results, example of a blocked release, and the defined structure-preservation metric/result.

8. Member 4 - Nancy: lightweight coding plus PowerPoint

Mission: prepare an accurate, convincing presentation while delivering one small, isolated coding artifact. Nancy is not expected to build a production-stage agent, detection model, evaluation framework, redactor or orchestrator.

Workload target: spend most effort on the deck (suggested planning split: 75% slides / 25% code). The code should be simple, deterministic, easy to test and independent from the live pipeline.

A. Small coding assignment: synthetic demo-fixture generator

Path	Implementation
demo_tools/generate_synthetic_samples.py	Generate a fixed set of invented .txt examples: a document with email/phone, a mixed-PII case, and a no-PII control. Use a fixed seed or fixed constants so outputs are reproducible.
demo_tools/README.md	Explain how to run the script, where outputs are written, what each scenario demonstrates, and why all values are synthetic.
tests/test_demo_fixtures.py	Check generation is deterministic; expected files and manifest are created; output is UTF-8; no network/API is called; and required scenario labels exist.
demo_samples/manifest.json (generated)	Include sample_id, filename, scenario description and synthetic=true. Do not add detection ground truth spans; Avishi owns annotations and evaluation.

Implementation steps

- Use only invented data such as Alex Example, alex@example.test and fictional phone/address values. Prefer reserved test domains and clearly artificial values.
- Keep the generator standalone: Python standard library only if possible. It must run from the repository root and must not import the production detector, redactor or UI.
- Write generated files to a documented demo output directory. Make reruns deterministic and avoid overwriting unrelated user files.
- Run the script, inspect each fixture manually, commit the small source/README/tests, and provide the generated examples to Tirth for the demo.
- Do not compute precision/recall/F1 or create entity-span ground truth; Avishi owns those. Do not redact files or integrate the main pipeline; Yash and Tirth own those parts.

B. PowerPoint responsibilities

- Create the slide theme and story early; use placeholders until actual test results exist.
- Explain the problem, scope, system architecture, module boundaries and why unsafe outputs are blocked.
- Collect Ayush's extraction evidence, Avishi's real benchmark metrics, Yash's leakage/structure tests and Tirth's integrated demo screenshots.
- Keep screenshots synthetic. Verify every number against the source test output; include sample size and limitations.
- Coordinate who speaks on each slide, rehearse the end-to-end demo, and prepare a backup screenshot/video.

Acceptance checklist

- The fixture script and tests run independently without secrets, network access or another member's code.
- The PPT accurately covers architecture, evidence, limitations, future work and demo flow.
- Her PR contains only the fixture helper, tests, documentation and demo assets; the deck can be updated separately.

9. Member 5 - Tirth: orchestration, UI, audit assembly and integration

Mission: connect public APIs, manage workflow state, build user interaction and assemble privacy-conscious audit/report output. Tirth is integration owner, not owner of the algorithms inside Modules 1-3.

Path	Responsibility
src/pii_guard/orchestration/pipeline.py	Call modules in agreed order using public interfaces only.
src/pii_guard/orchestration/state.py	Model RECEIVED, VALIDATED, EXTRACTED, DETECTED, REDACTED, VERIFIED, RELEASED, BLOCKED and FAILED.
src/pii_guard/orchestration/errors.py	Normalize safe error codes and catch timeouts/exceptions without converting failure into success.
src/pii_guard/reporting/audit.py	Assemble safe events: document ID, timestamp, statuses, counts, versions and error code.
src/pii_guard/reporting/report_generator.py	Export JSON and readable summary without source text or detected values.
app.py / frontend/	Upload, options, progress, safe counts, status, approved download and error display.
tests/test_integration.py	Use fake modules to test happy path, exceptions, incomplete processing and blocking.

Implementation recipe

- **Step 1 - mock-first:** build fake extractor/detector/redactor implementations that return the agreed schemas. The pipeline can be developed before teammates' production code is ready.
- **Step 2 - explicit state machine:** failed validation stops extraction; failed extraction stops detection; failed/incomplete verification blocks release.
- **Step 3 - public interfaces only:** call extract_document(), detect_pii() and redact_and_validate(). Do not import another member's internal helpers.
- **Step 4 - audit safely:** log IDs, timestamps, status, category counts, versions and safe error codes; never log full document text or matched values.
- **Step 5 - report:** create safe JSON first, then render a human-readable view from that same record. Keep report generation separate from detection logic.
- **Step 6 - UI:** provide upload, progress, safe category counts, verification status and error details. Hide/disable download unless release_allowed is exactly true.
- **Step 7 - integration tests:** test corrupt file, no detections, module exception, incomplete OCR, verification FAIL, verification INCOMPLETE and attempted blocked download.
- **Step 8 - handoff:** document one command to launch the app and one command to run all tests; pin dependencies and avoid laptop-specific paths.

Public API and report shape

```
def process_document(file_path: str, options=None) -> ProcessingResult:
    """Run pipeline, enforce release gate, return safe result/report."""

ProcessingReport = {
  "document_id": "doc-001",
  "workflow_status": "success|blocked|failed",
  "category_counts": {},
  "verification_status": "PASS|FAIL|INCOMPLETE",
  "created_at": "ISO-8601 timestamp",
  "module_versions": {}
}
```

Acceptance criteria

- Pipeline works with mocks independently of the production implementations.
- An unsafe, failed or incomplete output can never be downloaded through the UI/API.
- Audit/report output contains metadata only and can be generated repeatedly.
- Integration tests exercise both success and fail-closed paths.
- A new teammate can run the app using the setup document.

Independent test command

```
pytest -q tests/test_integration.py
```

10. Shared interface contracts - agree these before coding

The point of a contract is to let people work separately. These are illustrative dictionaries; in implementation, use typed Python dataclasses or Pydantic models if that matches the repository. Required keys and offset semantics must remain stable.

Contract A - ExtractedDocument

```
{
  "document_id": "doc-001",
  "source_type": "pdf|scanned_pdf|docx|pptx|txt",
  "text": "exact extracted text",
  "pages": [],
  "warnings": [],
  "extraction_status": "success|partial|failed"
}
```

Contract B - DetectionResult

```
{
  "document_id": "doc-001",
  "entities": [
    {"type": "EMAIL", "start": 12, "end": 29, "confidence": 0.98}
  ],
  "warnings": []
}
```

Contract C - RedactionResult

```
{
  "document_id": "doc-001",
  "artifact_ref": "private reference, not a public file path",
  "redactions_applied": 1,
  "verification_status": "PASS|FAIL|INCOMPLETE",
  "release_allowed": false,
  "findings": []
}
```

Contract D - ProcessingReport (Member 5)

```
{
  "document_id": "doc-001",
  "workflow_status": "success|blocked|failed",
  "category_counts": {},
  "verification_status": "PASS|FAIL|INCOMPLETE",
  "created_at": "ISO-8601 timestamp",
  "module_versions": {}
}
```

Contract rules

- Every module carries document_id so results cannot be mixed across requests.
- Entity offsets point into the exact string in ExtractedDocument.text; no downstream step silently normalizes it.
- The confidence field is a score produced by a detector, not a proof of correctness; document its scale/meaning.
- Avoid including entity_value or source snippets in normal logs/reports; only tests may use synthetic values.
- The sanitized artifact should be referenced by a controlled identifier/path. Do not expose arbitrary filesystem paths to the browser.
- Member 5 validates schemas at module boundaries and handles version mismatches as safe errors.

11. End-to-end sequence and failure handling

Step	Runtime action	Owner	Failure result
1	User uploads file and selects the allowed policy. API assigns document_id.	Tirth	Bad request => reject with safe message.
2	Check size/type/signature; quarantine input.	Ayush, called by Tirth	Invalid/corrupt/oversized => FAILED; no further processing.
3	Extract text, pages/slides and OCR warnings.	Ayush	No text or incomplete required OCR => FAILED/INCOMPLETE; do not release.
4	Detect pattern and context entities; normalize spans and categories.	Avishi	Detector failure => FAILED. Ambiguous detections are flagged, not concealed.
5	Apply redaction policy and write sanitized artifact.	Yash	Invalid/overlapping spans according to policy => safe error/block.
6	Re-open output; rescan; perform known-value and artifact checks.	Yash	FAIL/INCOMPLETE => BLOCKED; release_allowed=false.
7	Assemble audit metadata and report.	Tirth	Report error must not turn a blocked artifact into an approved one.
8	UI offers download only if gate passed. Nancy uses synthetic screenshot/results in slides.	Tirth / Nancy	Blocked response shows reason without exposing PII.

Threat and edge-case checklist

- Malicious or renamed file; unusually large file; corrupted document; password-protected or encrypted document.
- Scanned page with poor OCR; mixed scanned and text pages; rotated pages or table content.
- Overlapping entities, multiple PII types touching each other, punctuation variations, names indistinguishable from ordinary words.
- Text redacted in memory but left extractable in the exported artifact; cached original preview or stale download URL.
- Module timeout, schema change, missing optional NER dependency or audit/report failure.
- Attempt to download after FAIL/INCOMPLETE, repeated document ID, temp file left after error, real PII accidentally included in debug logs.

Required security default

When an essential step fails or cannot be checked, the safe default is BLOCKED. Do not silently skip verification, do not grant a public download URL before PASS, and do not send original text to a cloud AI model.

12. GitHub workflow: fork, branch, pull request and sync

Use a personal fork if you need an isolated copy or lack write access. If everyone has team-repository access, feature branches are simpler. In both cases, keep one shared integration branch (for example, develop) and merge through pull requests.

Option A - fork workflow

On GitHub, open the team's repository and click Fork. Then run:

```
git clone https://github.com/YOUR-USERNAME/REPOSITORY.git
cd REPOSITORY
git remote add upstream https://github.com/TEAM-OWNER/REPOSITORY.git
git fetch upstream
# Confirm 'develop' exists; otherwise use the agreed integration branch.
git checkout -b feature/member4-demo-presentation upstream/develop
git status
git add demo_tools/ tests/test_demo_fixtures.py presentation/
git commit -m "Add synthetic demo fixtures and presentation assets"
git push -u origin feature/member4-demo-presentation

# Open a PR:
# base = team repository / develop
# compare = your fork / feature/member4-demo-presentation
```

Option B - shared repository

```
git clone https://github.com/TEAM-OWNER/REPOSITORY.git
cd REPOSITORY
git checkout develop
git pull origin develop
git checkout -b feature/member4-demo-presentation
# edit only demo_tools/, its tests, and presentation assets
pytest -q tests/test_demo_fixtures.py
git add demo_tools/ tests/test_demo_fixtures.py presentation/
git commit -m "Add demo fixtures and presentation assets"
git push -u origin feature/member4-demo-presentation
# Open a PR from the feature branch into develop.
```

Member	Suggested branch	Owned paths
Ayush	feature/member1-ingestion	ingestion/, ingestion tests/docs
Avishi	feature/member2-detection	detection/, evaluation ground truth and metrics
Yash	feature/member3-redaction	redaction/, security/leakage tests
Nancy	feature/member4-demo-presentation	demo_tools/, demo-fixture tests/docs, presentation/
Tirth	feature/member5-integration	orchestration/, reporting/, frontend/, integration tests

To sync: git fetch upstream, then merge upstream/develop into your feature branch, rerun your module tests, and push. Replace placeholder URLs and branch names with those actually used by your team. Do not force-push a branch someone else is using.

13. Parallel-development plan: each member can start independently

Agree the shared schemas, offsets and release states once. Then members implement against their own fixtures/mocks and own directories. Integration starts through reviewed PRs; nobody needs to wait for another member's private implementation.

Phase	Ayush	Avishi	Yash	Nancy	Tirth
Kickoff	Validator + extraction schema	Taxonomy + entity schema	Policy + result schema	Slide outline + synthetic-demo spec	State machine + mock APIs
Build	Format adapters/OCR	Patterns + local NER adapter	Redactor + verifier	Small deterministic fixture generator + PPT outline	UI + audit/report structure
Test	Format/OCR/completeness	Ground truth + metrics	Leakage + preservation	Generator tests + screenshot-safe sample review	Mock pipeline + failure/block paths
Integrate	PR and fix own tests	PR and review own metrics	PR and fix security tests	Merge helper PR; give sample bundle + accurate slides	Connect public APIs + E2E tests
Demo	Explain extraction	Explain detection metrics	Explain redaction and release gate	Present; coordinate speaker handoffs	Run app and show PASS and blocked case

Mock-first pattern for Tirth

```
fake_extractor -> fixed ExtractedDocument
fake_detector -> fixed DetectionResult
fake_redactor -> fixed RedactionResult
fake_reporter -> fixed ProcessingReport

Replace each fake with the real public API after its PR merges.
```

Rules that reduce conflicts

- One owner per module folder; changes to shared contracts require a small reviewed PR.
- Nancy's generated samples are for demo/presentation only. Avishi maintains separate annotated ground truth; do not treat Nancy's manifest as evaluation labels.
- Keep generated outputs deterministic, synthetic and free of secrets. Only commit the small approved demo examples if the team agrees.
- Use public APIs and shared schemas, not private helpers. Run module tests before opening a PR.
- Protect main; integrate through pull requests and rerun end-to-end tests before the final demo.

14. Test strategy and definition of done

Layer	Owner	Examples
Unit tests	Each module owner	Valid/invalid input, schema, offsets, overlap, exceptions.
Extraction quality	Ayush	Per-format fixture output, OCR completeness and structure-retention measurement.
Detection benchmark	Avishi	Synthetic ground truth, exact-span/type matching, precision/recall/F1, FN review.
Leakage/security	Yash	Known test PII absent, saved artifact re-opened, FAIL/INCOMPLETE blocks release.
Integration	Tirth	Mock pipeline, exceptions, blocked download, report has no raw PII.
Presentation check	Nancy	Metrics tie to test outputs, synthetic screenshots, honest limitations.

Suggested commands

```
pytest -q tests/test_ingestion.py
pytest -q tests/test_detection.py
pytest -q tests/test_redaction.py
pytest -q tests/test_integration.py
pytest -q
```

Definition of done for technical members

- Implementation is pushed to the assigned feature branch and merged through a reviewed PR.
- Unit tests cover normal cases, edge cases and failures; the module is independently testable.
- Public API and schemas match docs/interfaces.md.
- No real PII, private source documents, credentials or API tokens are committed.
- Documentation explains setup, known limits and the test command.
- A one-page evidence handoff is given to Nancy.

15. Audit/privacy rules

- **Raw document:** restrict access and retention; never expose the original via the normal result/download path.
- **AI/NLP:** use local NLP where possible. Any later AI enrichment gets only approved redacted content and must be an explicitly documented feature.
- **Logs:** no raw text, passwords, tokens, full identifiers or entity_value fields.
- **Temporary files:** clear them after both successful and failed processing; tests should check cleanup if feasible.
- **Download gate:** require an explicit verified PASS; never equate a missing status with approval.
- **Evaluation:** use synthetic test data and clearly disclose dataset size, unsupported formats, OCR uncertainty and known misses.

16. Kickoff, final-demo checklist and final ownership

- Roster confirmed: Ayush Sinha, Avishi Sharma, Yash, Nancy and Tirth. Clarify Anshika's status if she is also participating.
- Shared integration branch exists; direct pushes to main are restricted.
- docs/interfaces.md defines the contracts and exact offset convention.
- Each member has an agreed branch/fork, owned directory and independent tests.
- Nancy's demo fixture generator creates deterministic invented samples and passes its own tests.
- Synthetic fixtures cover valid files, invalid files, no-PII input and known-PII cases.
- PASS is checked against the saved artifact; FAIL/INCOMPLETE blocks download.
- Audit/report output contains metadata only and no raw PII.
- All module tests and end-to-end tests pass; the demo shows one successful and one blocked case.
- Nancy's slides use actual test outputs, sample sizes and stated limitations.

Evidence handoff template - send to Nancy

Member and module | What was implemented | Test command and measured result/sample size | Success and failure example | Screenshot filename | Security limitation | PR/commit link

Member	Primary deliverable	Boundary
Ayush Sinha	Ingestion/OCR + extraction-quality tests	Not detection, redaction or UI
Avishi Sharma	Detection/classification + ground truth and metrics	Not parsing, redaction or UI
Yash	Redaction/verification/release gate + leakage/structure tests	Not detector internals or UI
Nancy	PowerPoint + lightweight synthetic demo generator, tests and docs	Not core algorithms, evaluation framework or orchestration
Tirth	Orchestration/UI + audit/report + integration tests	Not internal algorithms of Members 1-3

Definition of success: each member's module is independently testable, contracts stay stable, failures block unsafe release, demo inputs are synthetic, and the PowerPoint reflects measured outputs rather than unsupported claims.